

Section 1: Java Core, Collections & JVM Internals

Q1. Under what condition will the `equals()` and `hashCode()` contract break if a custom class uses a mutable field (like a non-final `id`) inside both methods?

- A. Only if the object is serialized and deserialized over a network.
- B. If the field is mutated *after* the object is added to a `HashSet` or as a key in a `HashMap`.
- C. If two different threads call `hashCode()` at the exact same millisecond.
- D. If the class also implements the `Comparable` interface.

Q2. Consider the following code snippet. What is the output?

java

```
String s1 = new String("DBS"); //====> brand new String object created in heap storage
String s2 = "DBS";
String s3 = s1.intern(); //====> Fetches the Pool reference for "DBS"
System.out.println((s1 == s2) + " " + (s2 == s3));
```

Use code with caution.

- A. false false
- B. true true
- C. false true
- D. true false

Q3. How does the `ConcurrentHashMap` achieve thread safety in Java 8+ without locking the entire map?

- A. It uses a single global `ReentrantLock` that applies exclusively to write operations.
- B. It segments the map into 16 fixed buckets and locks individual segments using `Segment<K, V>`.
- C. It uses `synchronized` on the first node of a bucket's bin and Compare-And-Swap (CAS) operations for empty bins.
- D. It makes all underlying array elements `volatile` and blocks all read operations during a write.

Q4. What is the behavioral difference between a `fail-fast` iterator and a `fail-safe` iterator?

- A. Fail-fast throws `ConcurrentModificationException` immediately; Fail-safe operates on a clone of the collection.
- B. Fail-fast operates on a clone; Fail-safe throws a `NullPointerException`.
- C. Fail-fast is used only in SQL databases; Fail-safe is used only in JVM memory.
- D. Fail-fast guarantees no data loss; Fail-safe guarantees complete data consistency.

Q5. When using Java Streams, what is the crucial difference between `map()` and `flatMap()`?

- A. `map()` is a terminal operation, while `flatMap()` is an intermediate operation.
- B. `map()` transforms each element into another object, while `flatMap()` flattens a stream of streams into a single stream.
- C. `map()` runs sequentially, while `flatMap()` automatically executes on a `ForkJoin` common pool parallelly.

- D. `map()` accepts a primitive predicate, while `flatMap()` requires an object Supplier.

Q6. Which Garbage Collector in Java is specifically designed for applications requiring large heaps (greater than 4GB) while maintaining predictable, ultra-low pause times?

- A. Serial Garbage Collector
- B. Parallel Garbage Collector (Throughput Collector)
- C. G1 (Garbage-First) or ZGC (Z Garbage Collector)
- D. CMS (Concurrent Mark Sweep) Collector

Q7. What happens if an exception is thrown inside a `try` block, and another exception is thrown inside the corresponding `finally` block?

- A. The exception from the `try` block is thrown; the `finally` exception is lost.
- B. The exception from the `finally` block suppresses and hides the exception from the `try` block.
- C. The JVM crashes immediately with a `FatalSystemError`.
- D. Both exceptions are combined into a `MultiException` wrapper automatically.

Q8. What is the primary purpose of the `java.lang.ref.WeakReference` class in Java?

- A. To protect objects from being modified by multi-threaded processes.
- B. To allow the Garbage Collector to reclaim an object when it is only weakly reachable, preventing memory leaks in caches.
- C. To establish a fast database connection string reference.
- D. To enforce compile-time optimization for immutable objects.

Q9. If a class overrides `equals(Object o)` but fails to override `hashCode()`, what is the most severe runtime consequence? original hashCode references the object reference are different for objects with same field value.

- A. The compiler will reject the code with a syntax error.
- B. Duplicate elements can be stored in a `HashSet`, breaking set guarantees.
- C. The JVM will leak memory rapidly via the PermGen/Metaspace area.
- D. All objects of this class automatically become immutable.

Q10. How does a Java Virtual Machine (JVM) handle the execution of `default` methods introduced in interfaces if a class implements two interfaces containing identical default method signatures?

- A. It compiles successfully and chooses the alphabetically first interface method.
- B. It throws a runtime `AbstractMethodError` when the class is loaded.
- C. It forces a compile-time error, requiring the developer to explicitly override and resolve the ambiguity.
- D. It delegates the execution to the parent abstract class automatically.

Section 2: Concurrency & Multithreading

Q11. What is the functional difference between `submit()` and `execute()` methods when using an `ExecutorService`?

- A. `submit()` is for `Runnable` tasks, while `execute()` is strictly for `Callable` tasks.
- B. `submit()` returns a `Future` object to track task status and results, while `execute()` returns `void`.
- C. `execute()` runs tasks asynchronously; `submit()` blocks the main thread until completion.
- D. `execute()` automatically spawns new physical OS threads, while `submit()` uses virtual fibers.

Q12. How does a `ThreadLocal` variable prevent data corruption in multi-threaded applications?

- A. By locking the shared variable across all threads using a mutex wrapper.
- B. By allocating an entirely independent copy of the variable for each individual thread.
- C. By converting the variable into an immutable block inside the Metaspace.
- D. By forcing all threads to read directly from the hardware's L1 cache pool.

Q13. In Java's concurrency framework, what problem does a `CountDownLatch` solve compared to a `CyclicBarrier`?

- A. A `CountDownLatch` cannot be reused once its count reaches zero, whereas a `CyclicBarrier` can be reset and reused.
- B. A `CountDownLatch` requires all threads to wait for each other at a common barrier point.
- C. A `CyclicBarrier` is used for database connection counts; `CountDownLatch` is for file I/O operations.
- D. A `CountDownLatch` completely eliminates deadlocks, while a `CyclicBarrier` only mitigates them.

Q14. What occurs during a "Deadlock" situation between two threads in Java?

- A. One thread crashes due to an unhandled `OutOfMemoryError`.
- B. Thread A holds Lock 1 and waits for Lock 2, while Thread B holds Lock 2 and waits for Lock 1; neither can proceed.
- C. Both threads execute the same exact line of code continuously in an infinite processing loop.
- D. The CPU temperature spikes because threads are yielding resources too quickly.

Q15. Why are atomic classes like `AtomicInteger` preferred over a standard synchronized block for basic increment counters (`count++`)?

- A. Atomic classes completely bypass the JVM and execute directly via native OS database kernels.
- B. They use non-blocking lock-free CPU instructions (Compare-And-Swap) which avoid the overhead of thread context switching.
- C. They guarantee that the application will consume 50% less physical memory.
- D. Synchronized blocks are deprecated in modern enterprise Java specifications.

Section 3: Spring Framework & Dependency Injection

Q16. What is the correct description of the default bean scope in Spring, and when is a default bean instantiated?

- A. Prototype scope; instantiated every single time it is injected into a class.
- B. Request scope; instantiated once per HTTP request-response cycle.
- C. Singleton scope; instantiated once per Spring ApplicationContext container initialization (eagerly).
- D. Session scope; instantiated once when a user authenticates.

Q17. How does Spring resolve a circular dependency between two singleton beans (Bean A injects Bean B, and Bean B injects Bean A) when using Setter Injection?

- A. It throws a fatal `BeanCurrentlyInCreationException` during startup.
- B. It uses a three-level cache mechanism (early singleton objects) to expose an uncompleted instance of Bean A.
- C. It automatically converts Bean B into a prototype scope bean at runtime.
- D. It forces the JVM to ignore the dependency and leaves the fields as `null`.

Q18. If you use Constructor Injection instead of Setter Injection for a circular dependency scenario (Bean A and Bean B), what happens?

- A. Spring handles it identically using the early singleton object cache map.
- B. The application fails to start up, throwing a `BeanCurrentlyInCreationException`.
- C. The container runs normally but skips initializing the dependent properties.
- D. A memory leak occurs instantly, consuming all heap allocations.

Q19. What is the primary difference between `@RestController` and `@Controller` annotations in Spring Boot?

- A. `@RestController` only handles HTTPS traffic, while `@Controller` handles standard HTTP traffic.
- B. `@RestController` automatically adds the `@ResponseBody` annotation to every method, converting return objects directly into JSON/XML.
- C. `@Controller` can only be used if the application is deployed on an external WebLogic server.
- D. `@RestController` disables security filters across all class endpoints automatically.

Q20. When utilizing Spring AOP, what is the functional purpose of a "JoinPoint"?

- A. The exact location in the physical database where two entities form a foreign key relationship.
- B. A specific point during execution—such as a method execution or exception handling—where advice can be applied.
- C. The network gateway port where microservices establish a secure handshake.
- D. A specialized thread-pooling configuration used for asynchronous tasks.

Section 4: Spring Boot, Transactions & JPA

Q21. If a public method marked with `@Transactional` calls another private method within the exact same class, how does Spring manage the transaction context?

- A. It creates a brand-new database transaction for the private method execution.
- B. The transaction configuration on the private method is completely ignored because Spring AOP proxies rely on public interface interception.
- C. It throws an `IllegalTransactionStateException` at execution runtime.
- D. It rolls back the outer public transaction automatically.

Q22. What does the `Isolation.READ_COMMITTED` level protect a database transaction from?

- A. Non-repeatable reads
- B. Phantom reads
- C. Dirty reads (reading uncommitted data changes from other concurrent transactions)
- D. All serialization anomalies entirely

Q23. In Spring Data JPA, what is the core difference between the `findById()` and `getReferenceById()` methods?

- A. `findById()` returns a lazy proxy object, while `getReferenceById()` hits the database instantly via an eager select query.
- B. `findById()` returns an `Optional` and hits the database instantly; `getReferenceById()` returns a lazy proxy without hitting the database until properties are accessed.
- C. `findById()` requires an active transaction context, while `getReferenceById()` can execute on disconnected instances.
- D. There is zero difference; `getReferenceById()` is simply an alias for `findById()`.

Q24. How does the Spring Boot Starters framework (`spring-boot-starter-*`) optimize project configuration?

- A. By pre-compiling all Java source code directly into optimized machine-level assembly patterns.
- B. By bundling common dependencies together and triggering auto-configuration classes based on classpath contents.
- C. By converting standard relational databases into in-memory caches upon deployment.
- D. By blocking external network traffic from accessing local development dependencies.

Q25. Which annotation allows you to dynamically inject a properties value defined inside the `application.properties` file directly into a Spring bean field?

- A. `@InjectProperty`
- B. `@Value("${property.name}")`

- C. `@EnvironmentVariable`
 - D. `@ConfigurationProperties(prefix = "all")`
-

Section 5: Microservices Architecture & Patterns

Q26. What primary problem does the "Circuit Breaker Pattern" (e.g., Resilience4j) solve within a distributed microservices network?

- A. It encrypts payloads traveling over open web domains.
- B. It prevents cascading failures across services by failing fast when a downstream service is unresponsive or degraded.
- C. It handles load-balancing between different nodes of an API Gateway.
- D. It converts synchronous REST communications into asynchronous Kafka messaging queues automatically.

Q27. When using Apache Kafka within a banking architecture, what mechanism guarantees that events are processed in the exact order they were produced?

- A. Specifying a globally unique topic name for every transaction.
- B. Ensuring that all related messages share the same **Partition Key**, sending them to the exact same Kafka partition.
- C. Setting the consumer pool count to exactly zero.
- D. Configuring the replication factor to match the broker node configuration.

Q28. What is the fundamental operational difference between the Saga Orchestration pattern and the Saga Choreography pattern?

- A. Orchestration relies on a central controller directing steps; Choreography uses decentralized event exchanges between services.
- B. Orchestration is exclusively synchronous; Choreography is exclusively local file I/O based.
- C. Orchestration does not support compensating transactions; Choreography enforces immediate rollbacks.
- D. Orchestration requires a relational database; Choreography relies entirely on NoSQL platforms.

Q29. What is the role of an API Gateway (like Spring Cloud Gateway) in a microservices ecosystem?

- A. To execute heavy internal business logic and run long-term data queries.
- B. To serve as a single entry point providing cross-cutting concerns like routing, authentication, rate limiting, and SSL termination.
- C. To replace individual service databases with a unified enterprise cluster.
- D. To compile front-end Angular/React bundles directly onto edge servers.

Q30. Why is the "JWT (JSON Web Token)" stateless, and how does a microservice verify its authenticity without querying an OAuth server every time?

- A. Because it stores data inside the browser's cookies securely.

- B. Because it contains a cryptographic signature signed by a private key; services verify it using the corresponding public key.
- C. Because it deletes itself automatically after a single read execution loop.
- D. Because it relies on hardware-level firewalls to guarantee secure token passage.

Answer Key & Core Technical Clarifications

Q#	Key	Core Technical Clarification
1	B	Modifying fields used in <code>hashCode()</code> changes the bucket index calculation. The object gets trapped in the wrong bucket, making it unretrievable and causing memory leaks.
2	C	<code>s1</code> is in heap storage; <code>s2</code> is in the String Pool. <code>s1.intern()</code> fetches the pool reference. Thus, <code>s1 == s2</code> is false (different memory addresses), but <code>s2 == s3</code> is true.
3	C	Java 8 eliminated segment-wide locks. It locks only the individual bucket head node during structural write conflicts, improving concurrent write throughput.
4	A	Fail-fast iterators modify the collection directly and use a <code>modCount</code> tracker. Fail-safe iterators safely iterate over a distinct snapshot copy of the data structure.
5	B	<code>map()</code> yields a stream of elements (1-to-1 conversion). <code>flatMap()</code> flattens a <code>Stream<List<T>></code> or <code>Stream<Stream<T>></code> down into a single <code>Stream<T></code> (1-to-many).
6	C	ZGC and G1 split the heap memory space into manageable independent regions to achieve predictable garbage collection pauses below 10 milliseconds.
7	B	The <code>finally</code> block execution is guaranteed. An exception thrown inside <code>finally</code> swallows any original exception raised inside the preceding <code>try</code> framework block.
8	B	If an object has no strong or soft references remaining, the Garbage Collector purges it immediately during the next cycle, regardless of active memory constraints.
9	B	Hashing collections rely on <code>hashCode()</code> to find the right bucket, and then <code>equals()</code> to find the match. Missing <code>hashCode()</code> means equal items map to different buckets.
10	C	The diamond problem forces compiler ambiguity. The developer must override the method explicitly or call <code>InterfaceName.super.methodName()</code> to resolve it.
11	B	<code>execute()</code> handles "fire-and-forget" tasks without returning feedback. <code>submit()</code> captures an executable context inside a handle-driven async <code>Future<T></code> architecture.
12	B	<code>ThreadLocal</code> holds an internal map keyed by the executing <code>Thread</code> . <code>Thread.currentThread()</code> . Thread data fields remain isolated from concurrent process threads.
13	A	Latches act as downstream gates that count down to zero and cannot be re-used. Barriers act as synchronization rendezvous points that cycle and reset cleanly.
14	B	Circular dependency on resource locks causes indefinite blocking. Neither thread can progress or yield because each is waiting for a lock held by the other.
15	B	CAS loops avoid putting threads into a blocked/waiting state. They instruct the underlying CPU hardware architecture to update values atomically if unchanged.
16	C	Spring components default to the <code>singleton</code> lifecycle scope. This instantiates a single instance eagerly at application launch to verify context validity.

- | | | |
|----|---|--|
| 17 | B | Spring uses 3 internal map levels. It exposes an unpopulated, raw object factory proxy early so dependent beans can wire their references before complete field injection. |
| 18 | B | Constructor injection forces immediate object instantiation. Since neither bean can be created without the other being fully built first, initialization fails. |
| 19 | B | <code>@RestController</code> combines <code>@Controller</code> and <code>@ResponseBody</code> . It completely eliminates the need to explicitly flag individual web endpoints with response mappers. |
| 20 | B | A <code>JoinPoint</code> is any target location (e.g., method boundary) capable of hosting AOP interceptions. The concrete code executed at this point is the "Advice". |
| 21 | B | Spring's transaction boundary enforcement uses AOP proxy wrappers. Internal class self-calls bypass this proxy layer entirely, leaving the target method untransacted. |
| 22 | C | Read Committed uses read locks or MVCC snapshots to ensure transactions only read data that has been successfully committed to disk by other users. |
| 23 | B | <code>findById()</code> hits the database immediately (<code>Eager</code>). <code>getReferenceById()</code> yields an unpopulated Hibernate proxy object, delaying database calls until property access. |
| 24 | B | Boot Starters match dependencies and scan conditionally using <code>@ConditionalOnClass</code> . If standard jars are present, it auto-configures the beans. |
| 25 | B | The <code>@Value</code> annotation evaluates expression strings using Spring Expression Language (SpEL) to pull key/value strings from application configuration files. |
| 26 | B | Circuit breakers watch down-stream dependency health metrics. When error thresholds cross the limit, the breaker opens, rejecting requests instantly to preserve resources. |
| 27 | B | Kafka guarantees message ordering within a single partition. Giving messages the same key ensures they go to the same partition and stay in order. |
| 28 | A | Orchestrator setups use a centralized master controller to issue steps. Choreography patterns rely on decentralized async message pub/sub loops. |
| 29 | B | An Edge Gateway acts as a single point of entry to unify common requirements like routing, metrics, auth verification, and rate limiting before hitting inner pods. |
| 30 | B | JWTs use cryptographic signatures (RSA/HMAC). The receiving service uses a public key to verify that the signature matches the token data without contacting the issuer. |